

Last updated: May 4, 2026 (Strapi 5 era) • 54 min read

# From Zero to Hero: Getting Started with GraphQL, Strapi and Next.js 16 Part 4

Strapi v5



Paul Bratslavsky  
May 4, 2026



Part 4 of a 4-part series on building with GraphQL, Strapi v5, and Next.js 16. Each part builds directly on the project from the previous post.

- **Part 1**, GraphQL basics with Strapi v5. Fresh install, a full Shadow CRUD tour, and your first custom resolvers.
- **Part 2**, Advanced backend customization. A `Note` + `Tag` model, middlewares and policies, custom queries, and custom mutations.
- **Part 3**, Next.js 16 frontend. Apollo Client on the App Router: Server Component reads, Server Action writes, one page per GraphQL operation.
- **Part 4 (this post)**, Users and per-user content. Sign-in, automatic ownership on every note, an `is-note-owner` policy for GraphQL writes, the gap that policy leaves on REST, and a Document Service middleware that closes both APIs from one file.

After Part 3 you have a note-taking app that anyone can use. In Part 4 every user signs in, sees only their own notes, and cannot read or change anyone else's, no matter which API they call.

## TL;DR

- Strapi's `users-permissions` plugin gives us JWT sign-in, a `User` content type, and the `register`, `login`, and `me` GraphQL operations. We turn it on and grant the right permissions. We do not write auth code ourselves.
- Notes get an `owner` relation to `User`. A **Document Service middleware** in `src/index.ts` `register()` writes `owner` automatically on every new note, taking the value from `ctx.state.user` (the signed-in user). The client cannot pass someone else's id; the middleware overwrites the field at the data layer. The same code path covers REST, GraphQL, custom resolvers, and the seed script.
- Reads work the same way. A second clause in the same middleware adds `owner = ctx.state.user.id` to the filter of every `findMany` and `findOne` on `Note`. REST `GET /api/notes` and GraphQL `Query.notes` both go through the Document Service, so one rule limits each user to their own notes on both APIs.
- Writes get two layers of protection. The read filter above means a non-owner cannot even load the target note, so any update or delete attempt against someone else's note misses. On top of that, an `is-note-owner` policy on the four GraphQL write mutations gives us a worked policy example and a second check before the resolver runs.
- The custom queries from Part 2 ( `searchNotes`, `noteStats`, `notesByTag` ) build their own filter object or query a different content type, so the Document Service middleware does not cover them. Each one gets an inline `owner` check. Part 4's Step 6.3 covers that.
- The frontend gets three small things: `/login` and `/register` pages backed by Server Actions that set an HTTP-only cookie, an Apollo link that adds `Authorization: Bearer <jwt>` to every request, and a `Next.js` `middleware.ts` that sends signed-out users to `/login`.

Backend authorization at a glance:

REST and GraphQL both end up calling `strapi.documents("api::note.note").<action>(...)`, so a rule placed there fires for both APIs.

## Prerequisites

- The Part 2 backend running on `http://localhost:1337/graphql`, with the `Note + Tag` schema, the soft-delete middlewares, the `cap-page-size` policy, and the test script ( `scripts/test-graphql.mjs` ) all green.
- The Part 3 frontend cloned and working against that backend, including the seeded notes and the home page rendering them. If you skipped Part 3, clone the starter template and run through Steps 1 through 5.
- Node.js 20.9+. Strapi 5 and Next.js 16 both require it.

## Scope

Part 4 covers backend authorization end-to-end and the minimum frontend wiring to test it. Skipped:

- **Social login (OAuth providers)**. Email + password only.
- **Password reset and email verification**. Not part of the authorization story. See the `users-permissions` docs when you need them.
- **Roles beyond owner / not-owner**. No admin, no read-only viewer, no sharing. The pattern below is the foundation; richer roles use the same machinery.
- **Note sharing or public links**. Same reason.
- **Refresh tokens**. The default 30-day JWT is fine for a tutorial. Production apps would issue a short-lived access token plus a refresh flow.

## How auth fits together

Three pieces work together so each user only sees their own notes:

**Sign-in (who you are).** Strapi's `users-permissions` plugin handles this for free. It ships a `User` content type, the `register` and `login` mutations, and a JWT issuer. Logging in returns `{ jwt, user }`. Every later request sends `Authorization: Bearer <jwt>`, the plugin verifies the signature, and the user object is available as `ctx.state.user` for the rest of the request.

**Authorization (what you can do).** Sign-in tells the server who the caller is. Authorization decides what they can see or change. The rule across Part 4 is simple: only the owner. Reads filter to `owner = ctx.state.user.id`. Writes only succeed if the target note's `owner.id` matches the caller's id.

**Ownership (the data that makes the check possible).** We add `owner` as a `manyToOne` relation from `Note` to `User`. The server assigns it automatically when a note is created. From then on, the authorization check is just `note.owner.id === user.id`.

The diagram in the TL;DR shows the request path. The Document Service middleware does most of the work: it writes `owner` on create, and it adds an `owner` filter on read. The `is-note-owner` policy on the four GraphQL write mutations is the worked policy example for Part 4 and a second check before the resolver runs. After Step 6 the policy is technically redundant on the GraphQL side (the Document Service middleware already keeps a non-owner from loading the target note when `Mutation.updateNote` runs), but we keep it on purpose so you end up with a working policy file in your codebase to compare.

## Step 1: Turn on sign-in and set the right permissions

`users-permissions` ships with every fresh Strapi project. Check that it is in `package.json`:

```
1 "@strapi/plugin-users-permissions": "^5.x"
```



### 1.1 Check the JWT secret

The plugin signs JWTs with a value from the `JWT_SECRET` environment variable. Open `.env` and make sure it has a real value:

```
JWT_SECRET=<a-long-random-string>
```



If it is missing, generate one with `openssl rand -hex 32` and paste it in. Anything 32 bytes or longer works. Keep it secret in production.

### 1.2 Lock down the Public role

After Part 2, anyone who can hit your server can read and write Notes and Tags. We need to close that. From now on, signed-out users can only **register** and **log in**. Everything else (listing notes, creating notes, updating notes) requires sign-in, and the rules in Steps 3 through 6 make sure each signed-in user only sees and edits their own data.

Open the admin UI at `http://localhost:1337/admin` → **Settings** → **Users & Permissions Plugin** → **Roles** → **Public**.

- Expand **Note** and **uncheck everything**. The Public role has no access to Notes.
- Expand **Tag** and leave `find` / `findOne` checked (tags are reference data, fine to expose). Uncheck `create`, `update`, `delete`.
- Expand **Users-permissions** → **Auth** and confirm `register` and `callback` are checked. They are usually enabled by default on the Public role; if either is unchecked, tick it now.

The other auth actions in that list ( `forgotPassword` , `resetPassword` , `emailConfirmation` , `sendEmailConfirmation` , `refresh` , `connect` ) are unused in this tutorial and can stay at whatever default Strapi shipped them with.

- Click **Save**.

Now go to **Roles** → **Authenticated**.

- Expand **Note** and check `find` , `findOne` , `create` , `update` . Leave `delete` unchecked.
- Expand **Tag** and check `find` and `findOne` . Anything else stays unchecked.
- Expand **Users-permissions** → **User** and confirm `me` is checked. It is enabled by default on the Authenticated role; the frontend will read it to display the signed-in user's name.
- Click **Save**.

### 1.3 Lock down the custom GraphQL mutations

In Part 2 we set `auth: false` on the three custom mutations ( `togglePin` , `archiveNote` , `duplicateNote` ) so they could be called without sign-in. That was fine while we had no users. Now we want them to require the same Authenticated role permissions you just set.

A bit of context on how the GraphQL plugin checks permissions:

- **Shadow CRUD operations** ( `Query.notes` , `Query.note` , `Mutation.createNote` , etc.) come pre-wired to a permission name. The list query `Query.notes` checks for `api::note.note.find` ; the single-item query `Query.note` checks for `api::note.note.findOne` ; `Mutation.createNote` checks for `api::note.note.create` . Those names match the checkboxes in the admin UI's Roles screen, so ticking `find` , `findOne` , `create` , and `update` on the Authenticated role in 1.2 is enough to cover both list and detail reads plus the basic writes.
- **Custom mutations** (the three from Part 2) are not pre-wired to anything. The plugin has no way to guess `togglePin` is "really an update". Without a setting that tells it which permission to check, the only check left is "must be signed in", and any signed-in user passes.

If you only deleted the `auth: false` lines, anonymous calls would still be blocked, but any signed-in user could call those mutations. The role checkboxes would not gate them. That is inconsistent with the Shadow CRUD behavior we just relied on.

The fix is to tell each custom mutation which permission to check, using the `auth.scope` setting in `resolversConfig` . Pinning and archiving are kinds of update, so they reuse `api::note.note.update` . Duplicating creates a new row, so it reuses `api::note.note.create` .

Open `src/extensions/graphql/mutations.ts` , scroll to the `resolversConfig` block at the bottom, and swap the three `auth: false` entries:

```
1 // before
2 "Mutation.togglePin": { auth: false },
3 "Mutation.archiveNote": { auth: false },
4 "Mutation.duplicateNote": { auth: false },
5
6 // after
7 "Mutation.togglePin": { auth: { scope: ["api::note.note.update"] } },
8 "Mutation.archiveNote": { auth: { scope: ["api::note.note.update"] } },
9 "Mutation.duplicateNote": { auth: { scope: ["api::note.note.create"] } },
```



What this gives us:

- **Anonymous requests** are rejected before the resolver runs. No JWT, no entry.
- **Signed-in users without the `update` permission** are also rejected. If you ever add a read-only "Viewer" role, those users cannot call `togglePin` or `archiveNote`. The role checkbox decides.
- **Signed-in users with `update`** get through, then run into the `is-note-owner` policy from Step 4, which checks that the user actually owns the row they are touching.

Three checks before the resolver runs: signed in, has the right permission, owns the row. The first two come from `auth.scope`. The third is the policy in Step 4.

**Same fix for the three custom queries.** Part 2 also added `searchNotes`, `noteStats`, and `notesByTag`, and they have the same `auth: false` entries in `src/extensions/graphql/queries.ts`. Open that file, scroll to the `resolversConfig` block at the bottom, and swap:

```
1 // before
2 // "Query.searchArticles": { auth: false },
3 "Query.searchNotes": { auth: false },
4 "Query.noteStats": { auth: false },
5 "Query.notesByTag": { auth: false },
6
7 // after
8 // "Query.searchArticles": { auth: false },
9 "Query.searchNotes": { auth: { scope: ["api::note.note.find"] } },
10 "Query.noteStats": { auth: { scope: ["api::note.note.find"] } },
11 "Query.notesByTag": { auth: { scope: ["api::note.note.find"] } },
```

`find` is the right scope because all three are reads. Signed-out calls are now rejected; signed-in calls go through.

Leave `Query.searchArticles` alone — it is a Part 1 holdover for the Article content type and is not part of the Note ownership story.

**What about scoping these to the current user?** Partially. The Document Service middleware in Step 6 adds an `owner` filter to every `findMany` and `findOne` on `api::note.note`, so `searchNotes` and `notesByTag` would inherit the scope through filter merging. The genuine gaps are inside `noteStats`: it calls `count()` (the middleware skips it because the early return only matches `findMany` and `findOne`) and `findMany` on `api::tag.tag` (the middleware's `uid` check skips other content types entirely). On top of that, we want each owner check visible at the call site so a future reader of the resolver does not have to know the middleware is silently merging filters in. Step 6.3 adds an explicit `owner` filter to all three resolvers for those reasons.

**Why reuse `update` instead of giving each mutation its own permission?** You could add a custom controller method for each mutation so it shows up as a separate checkbox in the admin UI. That would let an "Editor" role toggle pin without being able to do a full update. We do not need that level of detail here. There is one Authenticated role, every signed-in user has the same permissions, so reusing `update` and `create` keeps the file count down. The [Strapi GraphQL plugin docs](#) cover the custom-permission variant if you need it.

## 1.4 Test in the Sandbox

Open the Apollo Sandbox at <http://localhost:1337/graphql>. Same Sandbox you used in Parts 1 and 2.

**Check that the Public role cannot list notes.** Paste this and run it (no headers):

```
1 query {
2   notes {
3     documentId
4     title
```

```
5   }  
6 }
```

The response is `Forbidden` access. Good — signed-out reads are blocked before any middleware runs.

**Register a user.** We need an account to test with. Paste the mutation into the Operation editor first so the Sandbox knows the `$input` variable's type before you fill it in:

```
1 mutation Register($input: UsersPermissionsRegisterInput!) {  
2   register(input: $input) {  
3     jwt  
4     user {  
5       username  
6       email  
7     }  
8   }  
9 }
```



Then open the **Variables** panel at the bottom of the Sandbox and paste:

```
1 {  
2   "input": {  
3     "username": "testuser",  
4     "email": "testuser@example.com",  
5     "password": "testuser"  
6   }  
7 }
```



Click **Register**. The response has a JWT. Copy it — we will use it as a header in a moment.

**Logging in.** The JWT from `register` lasts 30 days, so you only register once per user. Every later session uses the `login` mutation. It takes `identifier` (username or email) and `password` and returns the same `{ jwt, user }` shape:

```
1 mutation Login($input: UsersPermissionsLoginInput!) {  
2   login(input: $input) {  
3     jwt  
4     user {  
5       username  
6       email  
7     }  
8   }  
9 }
```



And the Variables panel with:

```
1 {  
2   "input": {  
3     "identifier": "testuser",  
4     "password": "testuser",
```



```
5     "provider": "local"
6   }
7 }
```

`provider: "local"` is the built-in email-and-password provider. The input type requires it even though most apps never pass anything else. (OAuth flows would use `"google"` , `"github"` , etc.) Click **Login** and you get back a fresh JWT. The frontend in Step 8 calls this mutation when a returning user signs in.

**List notes as testuser.** Open the **Headers** panel and add a row. Header name is `Authorization` ; value is the word `Bearer` , a space, and the JWT you copied earlier. No quotes, no backticks:

| Key (Header Name) | Value                                                                          |
|-------------------|--------------------------------------------------------------------------------|
| Authorization     | Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiaWF0Ijox...rest-of-jwt |

- Type the literal word **Authorization** into the Key field — eight letters, no surrounding characters. The backticks you see in inline-code formatting elsewhere in this post are markdown styling on a rendered page, not part of the value.
- In the Value field, type **Bearer**, then a single space, then paste the JWT.
- No quotes, no backticks, no leading or trailing whitespace.
- Make sure the checkbox on the left of the row is ticked. Sandbox keeps unticked rows around but does not send them.

Now re-run the `notes` query:

```
1 query {
2   notes {
3     documentId
4     title
5   }
6 }
```



This time the response is *every note in the database*.

That is the problem Part 4 fixes. testuser is signed in, but the API does not know about ownership yet, so they see everyone's notes. Steps 3 through 6 close that gap.

If you would rather use the terminal, the same three checks via curl:

```
# 1. Public role: Forbidden
curl -s -X POST http://localhost:1337/graphql \
  -H 'Content-Type: application/json' \
  -d '{"query":"{ notes { documentId title } }"}'
# -> {"errors":[{"message":"Forbidden access", ... }], "data":null}

# 2. Register testuser
curl -s -X POST http://localhost:1337/graphql \
  -H 'Content-Type: application/json' \
  -d '{"query":"mutation R($input: UsersPermissionsRegisterInput!) { register(input: $input) }"}'
# -> {"data":{"register":{"jwt":"<JWT>", ... }}}

# 3. Signed-in read: every note in the database
JWT="<paste-the-token>"
curl -s -X POST http://localhost:1337/graphql \
```



```

-H 'Content-Type: application/json' \
-H "Authorization: Bearer $JWT" \
-d '{"query":{"notes { documentId title } }}"}'
# -> {"data":{"notes":[ ... every note in the database ... ]}}

```

## Step 2: Add the `owner` relation to Note

Open the admin UI → **Content-Type Builder** → **Note** → **Add another field**:

1. Pick **Relation**.
2. Leave **Note** on the left.
3. On the right, pick **User** (from: **users-permissions**).
4. Choose **manyToOne**: one user has many notes, each note has one owner.
5. Name the field **owner** on the Note side and **notes** on the User side.
6. Click **Finish**, then **Save**. Strapi restarts.

After restart, the Note schema has:

```

1 {
2   "owner": {
3     "type": "relation",
4     "relation": "manyToOne",
5     "target": "plugin::users-permissions.user",
6     "inversedBy": "notes"
7   }
8 }

```

The relation is nullable by default. Step 3 fixes that by stamping an owner automatically on every new note.

### 2.1 Backfill existing notes

The notes you seeded in Part 3 do not have an owner yet. The cleanest fix is a one-shot script that bootstraps Strapi, assigns every existing note to testuser, and exits. From `server/`, create `scripts/backfill-owner.js`:

```

1 // server/scripts/backfill-owner.js
2 "use strict";
3
4 async function main() {
5   const { createStrapi, compileStrapi } = require("@strapi/strapi");
6   const app = await createStrapi(await compileStrapi()).load();
7   app.log.level = "error";
8
9   const testuser = await app
10     .documents("plugin::users-permissions.user")
11     .findFirst({ filters: { username: "testuser" } });
12   if (!testuser) {
13     throw new Error('No user with username "testuser". Register one first.');

```



```

20   for (const note of notes) {
21     await app.documents("api::note.note").update({
22       documentId: note.documentId,
23       data: { owner: testuser.id },
24     });
25   }
26
27   console.log(
28     `Backfilled ${notes.length} notes to owner ${testuser.username}.`,
29   );
30
31   await app.destroy();
32   process.exit(0);
33 }
34
35 main().catch((err) => {
36   console.error(err);
37   process.exit(1);
38 });

```

Run it with `node scripts/backfill-owner.js`. The script boots a Strapi instance, runs the backfill against the Document Service, and exits. Or, if you prefer a GUI-based approach, open the admin UI's Content Manager and assign the owner relation on each note by hand.

Every existing note now has `testuser` as its owner. New notes get one automatically once Step 3 is in place.

### Step 3: Set the owner automatically on every new note

Clients should not be able to pick an owner. Every signed-in request has the user available as `ctx.state.user`; we want to use that to set `owner` on a new note no matter how the request comes in.

Strapi gives you four reasonable places to do this. Pick once and the same answer works for any "set this field on create" rule you hit later (`createdBy`, `tenantId`, `defaults`).

#### Option A: Content-type lifecycle hook

A `lifecycles.ts` file in `src/api/note/content-types/note/` exporting a `beforeCreate` function that modifies `event.params.data` before the row is written. This was the standard answer in Strapi v4.

Strapi v5 still supports it, but [Content lifecycle management](#) and [When to use lifecycle hooks in Strapi](#) explain why it is no longer the recommended option:

- It is buried inside a content-type folder, so a new developer has to know to look for it.
- It runs inside the database transaction, which makes error handling harder.
- There is no way to opt a single call out of the rule.

Skipping.

#### Option B: GraphQL middleware on `Mutation.createNote`

A `resolversConfig` middleware that reads `context.state.user` and writes `args.data.owner` before the resolver runs. Same setup as the `cap-page-size` policy from Part 2:

```

1 // hypothetical entry in src/extensions/graphql/middlewares-and-policies.ts
2 "Mutation.createNote": {
3   middlewares: [

```



```

4    async (next, parent, args, context, info) => {
5      const user = context?.state?.user;
6      if (user?.id) {
7        args.data = { ...(args.data ?? {}), owner: user.id };
8      }
9      return next(parent, args, context, info);
10   },
11 ],
12 },

```

Clean for GraphQL. But REST `POST /api/notes` does not run through this middleware. A client that sends `{"data": {"owner": 99, "title": "..."} }` to the REST endpoint writes `owner: 99` straight to the database. Same half-coverage problem we hit in Part 2.

### Option C: REST route middleware on the Note router

The mirror of Option B. A Koa middleware on `src/api/note/routes/note.ts` that writes `ctx.request.body.data.owner` before the controller runs. Covers `POST /api/notes`, does nothing for GraphQL. The gap is now on the other side.

You can do Options B and C together and cover both APIs, but then the rule lives in two files with two different argument names ( `args.data.owner` vs `ctx.request.body.data.owner` ). One side gets updated, the other does not, and the two slowly drift apart.

### Option D: Document Service middleware (what we use)

A **Document Service** middleware registered in `register()` runs whenever any code calls `strapi.documents("api::note.note").create(...)`. The Document Service sits below both REST and GraphQL: every controller and every GraphQL resolver eventually calls into it. So a rule here covers REST, GraphQL, custom resolvers (including `Mutation.duplicateNote`, which calls `strapi.documents("api::note.note").create(...)` directly), the seed script, and the admin UI, all from one file.

### Coverage

| Code Path                                                           | Lifecycle Hook | GraphQL Middleware | Route Middleware | Document Service Middleware |
|---------------------------------------------------------------------|----------------|--------------------|------------------|-----------------------------|
| REST <code>POST /api/notes</code>                                   | ✓              | ✗                  | ✓                | ✓                           |
| GraphQL <code>Mutation.createNote</code>                            | ✓              | ✓                  | ✗                | ✓                           |
| Custom resolver calling <code>strapi.documents(...).create()</code> | ✓              | ✗                  | ✗                | ✓                           |
| Seed script                                                         | ✓              | ✗                  | ✗                | ✓                           |
| Admin UI                                                            | ✓              | ✗                  | ✗                | ✓                           |
| Type-checked, visible at boot, opt-out-able per call                | ✗              | ✓                  | ✓                | ✓                           |
| Idiomatic Strapi v5                                                 | ✗              | ✓                  | ✓                | ✓                           |

### Why Option D

Two reasons.

**REST is always on in Strapi.** Even if your code only calls GraphQL, the REST routes still exist and accept requests. From the outside, every "GraphQL-only" Strapi app is also a REST API. A rule about how data gets written has to apply to both. Picking Option B and saying "we only use GraphQL" creates a hole that opens the first time a developer, a script, or an attacker sends a `POST /api/notes` request.

**Only Option D sits below both APIs.** Options B and C run inside one API's request handling: B inside the GraphQL resolver wrapper, C inside the Koa router. Option D runs inside the Document Service, which both APIs call into for every read and every write. A rule placed there cannot be bypassed by a new code path, because anything that touches Note data has to go through `strapi.documents("api::note.note").<action>(...)`.

So data rules (like "set this field automatically on create") go in Document Service middlewares. Step 3 puts owner-stamping there. Step 6 will add a second clause to the same middleware for read-scoping. One file holds both.

Open `server/src/index.ts` and register the middleware in `register()`. The Document Service middlewares docs say middlewares should be registered in the `register()` lifecycle, not `bootstrap()`, so the rule is in place before any plugin's bootstrap runs.

```
1 // server/src/index.ts
2 import type { Core } from "@strapi/strapi";
3 import registerGraphQLExtensions from "../extensions/graphql";
4
5 export default {
6   register({ strapi }: { strapi: Core.Strapi }) {
7     registerGraphQLExtensions(strapi);
8
9     // Stamp the authenticated user as `owner` on every new Note. Runs at
10    // the Document Service layer, so it applies to REST, GraphQL, custom
11    // resolvers, and seed scripts uniformly.
12    strapi.documents.use(async (context, next) => {
13      if (context.uid !== "api::note.note") return next();
14      if (context.action !== "create") return next();
15
16      const requestCtx = strapi.requestContext.get();
17      const user = requestCtx?.state?.user;
18      if (user?.id) {
19        context.params.data = {
20          ...(context.params.data ?? {}),
21          owner: user.id,
22        } as typeof context.params.data;
23      }
24
25      return next();
26    });
27  },
28
29  bootstrap() {
30    // Bootstrap intentionally empty for now. Step 6 leaves it that way too.
31  },
32 };
```

Three things to notice:

- **The middleware overwrites whatever the client sent.** Even if a client sends `data: { owner: 99 }` to `createNote` or `POST /api/notes`, the middleware runs at the data layer and writes `owner: <signed-in user's id>` instead. The client's `99` is dropped. The "I'll claim to be user 99" hole closes here, below the place either API hands the data off.
- **A signed-out create produces a note with no owner.** That is the right outcome, and Step 1 already blocks signed-out create through the role permissions before we ever reach the middleware. Two checks, one after the other.

- **Reads are unaffected.** The early `return next()` when `context.action !== "create"` makes the middleware skip on every other action ( `findMany` , `findOne` , `update` , `delete` ). Step 6 adds a second clause to the same middleware for read-scoping.

Restart Strapi and test in the Sandbox.

In the Sandbox at `http://localhost:1337/graphql` , make sure the `Authorization` header from Step 1.4 is still ticked. The two operations below show the middleware did its job without us having to read the `owner` field back through GraphQL.

Paste both into the Operation editor at once. The Sandbox supports multiple named operations in one document and shows a dropdown next to the run button:

```
1 query Me {
2   me {
3     id
4     username
5   }
6 }
7
8 mutation CreateNote {
9   createNote(
10    data: { title: "testuser owns me", content: "yo", pinned: false }
11  ) {
12    documentId
13    title
14  }
15 }
```

**Run Me first.** The response shows `username: "testuser"` . That is who the next mutation will be authenticated as, and the value the upcoming `MyNotes` query will filter by.

**Then run CreateNote .** The response contains a `documentId` and the `title` you sent. The `data` input did not include an `owner` field anywhere; the Document Service middleware stamped it server-side before the resolver wrote the row.

## List the current user's notes

We need one more verification: read back the notes belonging to the signed-in user and confirm the new note shows up.

**Try this query first.** Add this to the Sandbox document and run it (Authorization header still set):

```
1 query BrokenMyNotes($username: String!) {
2   notes(filters: { owner: { username: { eq: $username } } }) {
3     documentId
4     title
5   }
6 }
```

Variables:

```
1 { "username": "testuser" }
```

The response will be an error:

```
1 {
2   "errors": [
3     {
4       "message": "Invalid key owner",
5       "path": ["notes"],
6       "extensions": {
7         "code": "BAD_USER_INPUT",
8         "error": {
9           "name": "ValidationError",
10          "details": {
11            "key": "owner",
12            "path": "owner",
13            "source": "query",
14            "param": "filters"
15          }
16        }
17      }
18    },
19  ],
20  "data": null
21 }
```



Reading the stack trace from top to bottom tells you exactly what happened:

- **ValidationError: Invalid key owner** : Strapi rejects `owner` as a key the caller is not allowed to use *in this position* (the `filters` argument).
- **handleRegularRelation in @strapi/utils/.../throw-restricted-relations.js:87** : this function walks every relation named in `filters` and checks whether the caller has read permission on the related content type. `owner` points at User. The Authenticated role has no permissions on User. The check fails.
- **validateFilters → validateQuery → findMany** in the GraphQL plugin: the rejection happens *inside* the resolver. The Part 2 soft-delete middleware has already run by this point, and the database query has not yet been built.
- **resolversConfig.Query.notes.middlewares** at the bottom: our Part 2 soft-delete middleware. It is innocent here; the error fires after it.

The takeaway: **filtering on a relation requires read permission on the related content type, even if the response never selects any field from that relation.** Strapi enforces this during query validation, before the resolver runs the database query.

**The first instinct: grant the permission and move on.** It is right there in the admin UI. Open

<http://localhost:1337/admin> → **Settings** → **Users & Permissions Plugin** → **Roles** → **Authenticated** → expand **Users-permissions** → **User** → tick `find`. Save.

Re-run `BrokenMyNotes` in the Sandbox. The error is gone; the response is the array of notes whose `owner.username` equals `testuser`. Filter validated, query succeeded.

**Why this is the wrong fix.** That same `find` permission also unblocks the standard list endpoint for the User content type. With `testuser` still authenticated, run this in the Sandbox:

**Heads-up before running this.** If `testuser` is the only account you have registered so far, the response below will only contain one row and the privacy leak will not be obvious. Register a second user first (any username and email; reuse the `Register` mutation block from Step 1.4 with new variables, or sign in to the Sandbox with `testuser2 / testuser2 / testuser2@example.com`) so the query has more than one row to leak.

```
1 query AllUsers {
2   usersPermissionsUsers {
3     documentId
4     username
5     email
6   }
7 }
```



The response is **every user in the database**: `testuser`, `testuser2`, anyone else who registered, all their usernames and emails handed to any signed-in caller. The admin UI does not have a "let me filter by user, but don't let me list users" toggle. The `find` permission covers both. That is a real privacy leak.

**Go back to the admin UI and uncheck `find` on User for the Authenticated role.** Save. We need an approach that does not require the permission at all.

**The right approach: a custom `Query.myNotes` resolver.** It reads the user id from the JWT server-side, then filters using the Document Service. The Document Service builds the query directly against the database, so it does not run through the GraphQL validation that rejected our `owner` filter above. The client never names `owner` (the query takes no arguments), so the validation has nothing to reject. The User content type stays fully closed; signed-in callers cannot list users.

**Step 1: define a payload type and add the field.** Open `server/src/extensions/graphql/queries.ts`. We want `myNotes` to return *both* the signed-in user (so the frontend can render "logged in as") and the list of notes, in one round trip.

Add two new object types as siblings to the existing `TagCount` and `NoteStats` declarations at the top of the `types` array. The first, `MyNotesUser`, is a deliberately small view of the user that only exposes `id` and `username` (the only two fields a list page actually needs). The second, `MyNotesPayload`, wraps that user with the notes array:

```
1 nexus.objectType({
2   name: "MyNotesUser",
3   definition(t) {
4     t.nonNull.id("id");
5     t.nonNull.string("username");
6   },
7 },
8 nexus.objectType({
9   name: "MyNotesPayload",
10  definition(t) {
11    t.nonNull.field("user", { type: "MyNotesUser" });
12    t.nonNull.list.nonNull.field("notes", { type: "Note" });
13  },
14 },
```



**Why a custom `MyNotesUser` instead of reusing the built-in `UsersPermissionsMe` type?**

`UsersPermissionsMe` exposes `id`, `documentId`, `username`, `email`, `confirmed`, `blocked`, and `role`. If we returned that type from `myNotes`, a client could ask for any of those fields by writing them in their query (for example `myNotes { user { email role { name } } }`), and the server would return them. Since this query is

only meant to power a list page that needs `username` and nothing else, we say so in the schema itself.

`MyNotesUser` has two fields, and those are the only two fields a client of `myNotes` can ever ask for. The schema is the contract; relying on the client to "select less" is not.

If a page genuinely needs `email`, `role`, or other user fields (think a profile or settings page), it calls `Query.me` directly on that page. Two queries, two purposes, each one returning only the fields its page actually renders.

Then inside the existing `nexus.extendType({ type: "Query", definition(t) { ... } })` block, after the `notesByTag` field, add the resolver:

```
1 t.field("myNotes", {
2   type: nexus.nonNull("MyNotesPayload"),
3   async resolve(_parent: unknown, _args: unknown, ctx: any) {
4     const user = ctx?.state?.user;
5     if (!user?.id) {
6       // Should never reach here because of the `auth.scope` config below,
7       // but a tutorial-grade fallback. Return an empty payload shape that
8       // the frontend can render without special cases.
9       return { user: { id: 0, username: "" }, notes: [] };
10    }
11    const notes = await strapi.documents("api::note.note").findMany({
12      filters: { owner: { id: user.id } },
13      sort: ["pinned:desc", "updatedAt:desc"],
14      populate: ["tags"],
15    });
16    return {
17      user: { id: user.id, username: user.username },
18      notes,
19    };
20  },
21 });
```

The resolver explicitly picks only `id` and `username` off `ctx.state.user` and assembles the slim payload. Even if a future change adds new fields to `MyNotesUser`, the resolver only sends what it explicitly chooses to.

**Step 2: tie it to the role permission.** In the same file's `resolversConfig` block at the bottom, add:

```
1 "Query.myNotes": { auth: { scope: ["api::note.note.find"] } },
```

This requires the caller to have `find` on `Note` (which the `Authenticated` role already has from Step 1.2). Anonymous calls are rejected before the resolver runs.

**Step 3: save and let Strapi restart.** The dev server picks up the change automatically.

**Step 4: run the working query.** Replace the broken `BrokenMyNotes` operation in the Sandbox with:

```
1 query MyNotes {
2   myNotes {
3     user {
4       id
5       username
6     }
7     notes {
8       documentId
9       title
10      pinned
11    }
12  }
13 }
```

```
12   }  
13 }
```

The response contains the current user's `id` and `username` plus the array of notes owned by them, including the one `CreateNote` just created. Run it as `testuser2` (different JWT) and the `user` block changes to `testuser2` and the `notes` array contains only their notes.

#### What this resolver gets right:

- **The user id stays on the server.** The client never sees, stores, or passes it. The JWT is the source of truth, and the server is the only thing that reads it.
- **No new role permissions needed.** The User content type stays closed, and no usernames or emails leak to other signed-in users.
- **The schema only exposes two fields on the user.** A client of `myNotes` can only ever ask for `id` and `username`. There is no way to write a query that pulls back `email` or `role` from this endpoint, even if a future change to the resolver accidentally returns more.
- **One endpoint, zero client logic.** A page that wants "the signed-in user's notes" calls `myNotes` and is done. There is no path where the client could pass a wrong id, because the client does not pass an id at all.

**Looking ahead: Document Service middleware vs custom resolver.** A custom resolver like `myNotes` is the right tool when you want a *named* endpoint that the schema makes obvious (a query called `myNotes` is a clear contract: "the current user's notes"). A Document Service middleware is the right tool when you want the *existing* endpoint to behave correctly without any per-call work from the caller (the bare `notes` query should always be scoped, no matter who calls it from where). Step 6 adds a Document Service middleware so the bare `notes` query also returns only the caller's notes, on both REST and GraphQL. Neither approach replaces the other; we use both in the final design.

## Step 4: A first cut at the `is-note-owner` policy

In Part 2 we said the real authorization example would land in Part 4. Here it is.

Only the note's owner should be able to update it. That means gating these four mutations, all of which take a note's `documentId` as input:

- `Mutation.updateNote`
- `Mutation.togglePin`
- `Mutation.archiveNote`
- `Mutation.duplicateNote`

`Mutation.createNote` is intentionally excluded; ownership is *set* there by the Document Service middleware from Step 3, not *checked*.

Create `server/src/policies/is-note-owner.ts`:

```
1 // server/src/policies/is-note-owner.ts  
2 import type { Core } from "@strapi/strapi";  
3  
4 type PolicyContext = {  
5   args?: { documentId?: string };  
6   state?: { user?: { id?: number | string } };  
7   context?: { state?: { user?: { id?: number | string } } };  
8 };  
9
```





```

10 const isNoteOwner = async (
11   policyContext: PolicyContext,
12   _config: unknown,
13   { strapi }: { strapi: Core.Strapi },
14 ): Promise<boolean> => {
15   const user =
16     policyContext?.state?.user ?? policyContext?.context?.state?.user;
17   if (!user?.id) {
18     strapi.log.warn("is-note-owner: rejected, no authenticated user.");
19     return false;
20   }
21
22   const documentId = policyContext?.args?.documentId;
23   if (!documentId) {
24     strapi.log.warn("is-note-owner: rejected, no documentId in args.");
25     return false;
26   }
27
28   const note = await strapi
29     .documents("api::note.note")
30     .findOne({ documentId, populate: ["owner"] });
31
32   if (!note) return false;
33   if (note.owner?.id === user.id) return true;
34
35   strapi.log.warn(
36     `is-note-owner: rejected, user ${user.id} is not the owner of note ${documentId}.`,
37   );
38   return false;
39 };
40
41 export default isNoteOwner;

```

Now wire the policy onto the four write mutations.

Open `server/src/extensions/graphql/middlewares-and-policies.ts` and find the `resolversConfig` block. It already has entries for `Query.notes` and `Query.note` from Part 2. Add four new sibling keys inside the same object, right after the `Query.note` block:

```

1  return {
2    resolversConfig: {
3      "Query.notes": {
4        // ... unchanged from Part 2 ...
5      },
6      "Query.note": {
7        // ... unchanged from Part 2 ...
8      },
9
10     // NEW: per-row ownership check on every Note write
11     "Mutation.updateNote": { policies: ["global::is-note-owner"] },
12     "Mutation.togglePin": { policies: ["global::is-note-owner"] },
13     "Mutation.archiveNote": { policies: ["global::is-note-owner"] },
14     "Mutation.duplicateNote": { policies: ["global::is-note-owner"] },
15   },
16 };

```

A few things to know:

- The four new keys go inside the same `resolversConfig` object as `Query.notes` and `Query.note`. Same indentation, separated by commas.

- `Mutation.createNote` is **not** in the list. Ownership is *set* on create by the Document Service middleware from Step 3, not *checked*.
- All four mutations take `documentId` as an argument. That is what the policy reads to load the target row.
- The string `"global::is-note-owner"` matches the filename `src/policies/is-note-owner.ts` you wrote above. Strapi auto-registers files in `src/policies/` under that prefix; if the file is missing or misnamed, Strapi will fail to start with `Cannot find policy is-note-owner`.

## Test the policy in the Sandbox

We need a second user to play the part of "not the owner." If you already registered one earlier (for the privacy-leak demonstration in Step 3), reuse the `Login` mutation block from Step 1.4 to grab a fresh JWT for that account, then jump down to the `TogglePin` test. Otherwise, register `testuser2` now in the Sandbox:

```
1 mutation Register($input: UsersPermissionsRegisterInput!) {
2   register(input: $input) {
3     jwt
4     user {
5       username
6     }
7   }
8 }
```



Variables:

```
1 {
2   "input": {
3     "username": "testuser2",
4     "email": "testuser2@example.com",
5     "password": "testuser2"
6   }
7 }
```



Copy testuser2's JWT.

Now grab any existing note's `documentId` (they all belong to testuser after the backfill in Step 2.1). Swap the Authorization header in the Sandbox to `Bearer <testuser2-jwt>` and try to pin testuser's note:

```
1 mutation TogglePin($id: ID!) {
2   togglePin(documentId: $id) {
3     documentId
4     pinned
5   }
6 }
```



Variables:

```
1 { "id": "<paste-note-documentId>" }
```



The response is `Policy Failed`.

Swap the header back to testuser's JWT and run the same mutation. Now it succeeds and the note flips between pinned and unpinned.

## Step 5: The REST leak

Anything in `resolversConfig` only runs for GraphQL. The same gap we hit in Part 2 with the soft-delete and page-cap rules. The `is-note-owner` policy does nothing for REST, and testuser2 can prove it by editing testuser's note over REST:

```
TESTUSER2_JWT=<testuser2-jwt>
NOTE_ID=<a-note-documentId>
curl -s -X PUT "http://localhost:1337/api/notes/$NOTE_ID" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $TESTUSER2_JWT" \
  -d '{"data":{"title":"testuser2 was here"}}'
# -> 200 OK, title updated.
```

That is a real bug. The policy gates GraphQL but not REST. One rule, half-covered.

Restore the title before moving on:

```
TESTUSER_JWT=<testuser-jwt>
curl -s -X PUT "http://localhost:1337/api/notes/$NOTE_ID" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $TESTUSER_JWT" \
  -d '{"data":{"title":"<original title>"}}'
```

Two reasonable ways to close the gap:

1. **Mirror the rule on REST.** Add a route-level middleware on `src/api/note/routes/note.ts` (same setup Part 2 used for soft-delete) plus a small policy on the route's `find`, `findOne`, `update`, and `delete` actions that loads the target note and rejects when the owner does not match. The rule now lives in two files: a `resolversConfig` policy for GraphQL and a route middleware/policy for REST. Both have to be kept in sync.
2. **Move the rule below both APIs into a Document Service middleware.** Step 3 already uses one to write `owner` on create. We extend that same middleware with a second clause: it watches every `findMany` and `findOne` on `api::note.note` and adds `owner = ctx.state.user.id` to `params.filters.REST` controllers and GraphQL Shadow CRUD resolvers both eventually call `strapi.documents("api::note.note").<action>(...)`, so a rule placed there fires for both. One file, both APIs.

The next step builds option 2.

**Why not option 1?** It works, but it puts the same rule in two files. A future change updates the GraphQL side and forgets the REST side, and the same kind of leak we just demonstrated comes back. Option 2 puts the rule one layer deeper, where REST and GraphQL share the same code path: any caller that ends up at `strapi.documents("api::note.note").findOne(...)` goes through the rule, with no extra wiring. We took a similar position in Part 2's "GraphQL-only vs. both APIs" section. Part 4's ownership rule is the case where that one-rule-one-file approach earns its keep, because the rule has to read `ctx.state.user` and behave the same way for REST and GraphQL.

## Step 6: A Document Service middleware for both APIs

The owner-stamping middleware from Step 3 already lives in `src/index.ts` `register()`. We add two more `strapi.documents.use(...)` clauses in the same file, sitting right after it.

The three clauses together do three things:

1. **Stamp owner on every new note** (the existing Step 3 clause).
2. **Filter `findMany` and `findOne` to the signed-in user.** Both REST `GET /api/notes /:id` and GraphQL `Query.notes / Query.note` go through the Document Service, so one rule covers both.
3. **Block update and delete for non-owners.** A pre-flight `findOne` (which inherits the read filter from clause 2) returns null for non-owners; we throw `NotFoundError`. REST `PUT / DELETE /api/notes/:id` is covered the same way GraphQL `updateNote / deleteNote` is.

The rules live at the Document Service layer, so REST and GraphQL share them. There is no per-resolver attachment on the GraphQL side and no route-level middleware on the REST side. One file, both APIs, both verbs.

## 6.1 Extend the Document Service middleware

Open `server/src/index.ts` and replace its contents:

```
1 // server/src/index.ts
2 import type { Core } from "@strapi/strapi";
3 import { errors } from "@strapi/utils";
4 import registerGraphQLExtensions from "../extensions/graphql";
5
6 export default {
7   register({ strapi }: { strapi: Core.Strapi }) {
8     registerGraphQLExtensions(strapi);
9
10    // From Step 3: stamp `owner` on every new note.
11    strapi.documents.use(async (context, next) => {
12      if (context.uid !== "api::note.note") return next();
13      if (context.action !== "create") return next();
14
15      const requestCtx = strapi.requestContext.get();
16      const user = requestCtx?.state?.user;
17      if (user?.id) {
18        context.params.data = {
19          ...(context.params.data ?? {}),
20          owner: user.id,
21        } as typeof context.params.data;
22      }
23
24      return next();
25    });
26
27    // Scope every Note read to the signed-in user.
28    strapi.documents.use(async (context, next) => {
29      if (context.uid !== "api::note.note") return next();
30      if (context.action !== "findMany" && context.action !== "findOne") {
31        return next();
32      }
33
34      const requestCtx = strapi.requestContext.get();
35      const user = requestCtx?.state?.user;
36      const existingFilters = (context.params.filters ?? {}) as Record<
37        string,
38        unknown
39      >;
40
41      if (!user?.id) {
42        // No signed-in user: force the filter to match nothing.
```

```

43     // -1 is never a valid User id, so the query returns zero rows
44     // instead of falling through to "return everything."
45     context.params.filters = {
46       ...existingFilters,
47       owner: { id: { $eq: -1 } },
48     } as typeof context.params.filters;
49     return next();
50   }
51
52   context.params.filters = {
53     ...existingFilters,
54     owner: { id: { $eq: user.id } },
55   } as typeof context.params.filters;
56   return next();
57 });
58
59 // Gate every Note write (update, delete) for non-owners.
60 // The pre-flight findOne goes back through the Document Service
61 // and inherits the read filter above, so a non-owner sees null
62 // for the lookup. We throw NotFoundError to match the read
63 // posture (no existence leak via 403).
64 strapi.documents.use(async (context, next) => {
65   if (context.uid !== "api::note.note") return next();
66   if (context.action !== "update" && context.action !== "delete") {
67     return next();
68   }
69
70   const documentId = (context.params as { documentId?: string })
71     .documentId;
72   if (!documentId) return next();
73
74   const existing = await strapi
75     .documents("api::note.note")
76     .findOne({ documentId });
77   if (!existing) {
78     throw new errors.NotFoundError("Note not found.");
79   }
80
81   return next();
82 });
83 },
84
85 bootstrap() {},
86 };

```

What this code does.

**On reads ( `findMany` and `findOne` ):**

- **It looks up who is signed in.** Inside this middleware we do not get the request directly, so we ask Strapi for it with `strapi.requestContext.get()`. If `testuser` is signed in, `user.id` is `testuser`'s id. If nobody is signed in, `user` is `undefined`.
- **If nobody is signed in, the query returns zero notes.** We add the filter `owner.id = -1` to the query. No user has id `-1`, so the database finds nothing. We do not throw an error here because the route in Step 1 already rejects requests without a JWT. This is just a backstop in case anything slips past it.
- **The caller cannot ask for someone else's notes.** If a caller sends `notes(filters: { owner: { id: { eq: 99 } } })` hoping to read user 99's notes, we copy their filter into ours, then write our own `owner` value on top. Our value wins, so the database only returns notes owned by the signed-in user.

**On writes ( `update` and `delete` ):**

- **It checks ownership by looking the note up first.** Before the update or delete runs, we call `strapi.documents("api::note.note").findOne({ documentId })`. That lookup goes through the same middleware, so the read code above runs on it and adds `owner = signed-in user` to the query. If testuser2 tries to update one of testuser's notes, the lookup returns `null`, and we throw. We never write the ownership check twice; the write code borrows it from the read code.
- **It throws `NotFoundError`, not `ForbiddenError`.** When testuser2 reads testuser's note, the read code above already returns nothing, as if the note does not exist. We want the write to look the same. `ForbiddenError` would tell testuser2 "this note exists, you just cannot touch it," which tells them something they should not know.
- **It only runs on `update` and `delete`.** `create` is already handled by the owner-stamping code at the top of the file. Plain reads are handled by the read code above. `count` is handled inside `noteStats` in Step 6.3.

**Heads-up: this fires on the admin UI too.** The Document Service sits below every API surface, including the Content Manager in `http://localhost:1337/admin`. When you open Note in the admin UI, the request runs as your admin user (a row in `admin::user`), and `strapi.requestContext.get()?.state?.user` returns that admin user. The owner filter then becomes `owner.id = <admin-user-id>`, but `owner` points at `plugin::users-permissions.user`, which is a different content type. The two id spaces happen to overlap by coincidence, so the Content Manager's Note list will look mostly empty (or show the wrong rows) until you log in as the end-user whose id matches an admin id. If you need the Content Manager to keep working as an admin, branch on `requestCtx?.state?.auth?.strategy` and `return next()` early when the strategy is the admin one ( `"admin"` ) so admins keep their full view; the rule we wrote above only applies to the public REST and GraphQL surfaces. The tutorial leaves the simpler version in place because the demo never opens the Note Content Manager again, and an admin caveat hides the rule we just spent the section building.

## 6.2 Test it

Restart Strapi, then exercise reads and writes on both APIs.

**GraphQL reads.** With testuser's JWT in the Authorization header, run:

```
1 query {
2   notes {
3     documentId
4   }
5 }
```



You see the notes testuser owns. Swap the Authorization header to testuser2's JWT and re-run. testuser2 sees an empty list (or only the notes they created themselves).

**GraphQL writes.** Try to update one of testuser's notes as testuser2:

```
1 mutation UpdateNote($id: ID!, $data: NoteInput!) {
2   updateNote(documentId: $id, data: $data) {
3     documentId
4     title
5   }
6 }
```



Variables:

```
1 {
2   "id": "<testuser-note-documentId>",
3   "data": { "title": "testuser2 was here" }
4 }
```



The response is `Policy Failed`. The `is-note-owner` policy from Step 4 rejects the call before the resolver runs. Even if the policy were absent, the write-gate clause would now throw `NotFoundError` from the same lookup.

**REST reads.** Same shape, different surface:

```
TESTUSER_JWT=<testuser-jwt>
TESTUSER2_JWT=<testuser2-jwt>

curl -s -H "Authorization: Bearer $TESTUSER_JWT" "http://localhost:1337/api/notes" | jq '.data'
# -> 9

curl -s -H "Authorization: Bearer $TESTUSER2_JWT" "http://localhost:1337/api/notes" | jq '.data'
# -> 0
```

**REST writes.** This is what the write-gate clause is for. Pick any of testuser's note `documentId`s and try to PUT to it as testuser2:

```
NOTE_ID=<testuser-note-documentId>

curl -s -X PUT "http://localhost:1337/api/notes/$NOTE_ID" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $TESTUSER2_JWT" \
  -d '{"data":{"title":"testuser2 was here"}}'
# -> {"data":null,"error":{"status":404,"name":"NotFoundError", ... }}
```

testuser2's PUT returns 404. testuser's own PUT on the same note returns 200 and updates the title.

```
curl -s -X PUT "http://localhost:1337/api/notes/$NOTE_ID" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $TESTUSER_JWT" \
  -d '{"data":{"title":"testuser updated this"}}'
# -> 200, title now "testuser updated this"
```

One file ( `src/index.ts` ), three clauses, both APIs, both verbs. Reads scoped, writes gated, ownership stamped on create. The intro promise — *a Document Service middleware that closes both APIs from one file* — holds.

### 6.3 Scope the three custom queries to the owner

The Document Service middleware covers `Query.notes`, `Query.note`, and the REST `find/findOne`, because all four end up calling `strapi.documents("api::note.note").findMany(...)` or `findOne(...)`. The three custom queries from Part 2 ( `searchNotes`, `noteStats`, `notesByTag` ) are different:

- `searchNotes` and `notesByTag` call `strapi.documents("api::note.note").findMany(...)` themselves, with their own filter object. The middleware's `owner` clause does merge in, but it is harder to read what the final filter is from inside the resolver, and any future change to the resolver could clobber it. Adding the `owner` clause inline makes the rule visible at the call site.
- `noteStats` calls `count()` three times (which the middleware does not match because the early return only allows `findMany` and `findOne`) and then `findMany` on `api::tag.tag` (which is a different content type, so the middleware's `uid` check skips it entirely).

Step 1.3 already made all three queries require sign-in. We now add an explicit `owner` filter inside each resolver so each signed-in user only sees their own data.

The same pattern in all three: `read ctx.state.user.id` and `add owner: { id: user.id }` to the filter. Each resolver now takes `ctx` as its third argument.

Open `server/src/extensions/graphql/queries.ts`.

`searchNotes`. Replace the resolver with:

```
1 t.list.field("searchNotes", {
2   type: nexus.nonNull("Note"),
3   args: {
4     query: nexus.nonNull(nexus.stringArg()),
5     includeArchived: nexus.booleanArg({ default: false }),
6   },
7   async resolve(
8     _parent: unknown,
9     { query, includeArchived }: { query: string; includeArchived: boolean },
10    ctx: any,
11  ) {
12    const user = ctx?.state?.user;
13    if (!user?.id) return [];
14    const where: any = {
15      title: { $containsi: query },
16      owner: { id: user.id },
17    };
18    if (!includeArchived) where.archived = false;
19    return strapi.documents("api::note.note").findMany({
20      filters: where,
21      populate: ["tags"],
22      sort: ["pinned:desc", "updatedAt:desc"],
23    });
24  },
25 });
```

`noteStats`. Each `count` and `findMany` call gets the owner filter:

```
1 t.nonNull.field("noteStats", {
2   type: "NoteStats",
3   async resolve(_parent: unknown, _args: unknown, ctx: any) {
4     const user = ctx?.state?.user;
5     if (!user?.id) return { total: 0, pinned: 0, archived: 0, byTag: [] };
6     const owner = { id: user.id };
7     const [total, pinned, archived, tags] = await Promise.all([
8       strapi.documents("api::note.note").count({ filters: { owner } }),
9       strapi.documents("api::note.note").count({
10         filters: { pinned: true, owner },
11       }),
12       strapi.documents("api::note.note").count({
13         filters: { archived: true, owner },
14       }),
15       strapi.documents("api::tag.tag").findMany({
16         populate: { notes: { filters: { owner } } },
17         sort: ["name:asc"],
18       }),
19     ]);
20
21     const byTag = tags
22       .map((tag: any) => ({
23         slug: tag.slug,
24         name: tag.name,
25         count: Array.isArray(tag.notes) ? tag.notes.length : 0,
26       }));
```



```

27     .sort((a, b) => b.count - a.count || a.name.localeCompare(b.name));
28
29     return { total, pinned, archived, byTag };
30   },
31 });

```

The line worth pointing at is `populate: { notes: { filters: { owner } } }`. Every user can see every tag (tags are shared), but the *number of notes* on each tag has to be scoped to the current user. The Document Service lets us filter populated relations inline, so we add `{ owner }` to the `populate`. Each user sees their own per-tag counts, never anyone else's.

`notesByTag` .Add `owner` to the existing filter object:

```

1  t.list.field("notesByTag", {
2    type: nexus.nonNull("Note"),
3    args: { slug: nexus.nonNull(nexus.stringArg()) },
4    async resolve(_parent: unknown, { slug }: { slug: string }, ctx: any) {
5      const user = ctx?.state?.user;
6      if (!user?.id) return [];
7      return strapi.documents("api::note.note").findMany({
8        filters: {
9          archived: false,
10         tags: { slug: { $eq: slug } },
11         owner: { id: user.id },
12       },
13       populate: ["tags"],
14       sort: ["pinned:desc", "updatedAt:desc"],
15     });
16   },
17 });

```

Save. Strapi auto-restarts.

**Test in the Sandbox.** With `testuser`'s JWT in the Authorization header, run each one:

```

1  query {
2    searchNotes(query: "GraphQL") {
3      documentId
4      title
5    }
6  }

```

```

1  query {
2    noteStats {
3      total
4      pinned
5      archived
6      byTag {
7        slug
8        name
9        count
10     }
11   }
12 }

```

```

1 query {
2   notesByTag(slug: "ideas") {
3     documentId
4     title
5   }
6 }

```



You only see testuser's notes. Swap the Authorization header to testuser2's JWT and re-run the same queries. testuser2 only sees their own notes.

**Why repeat the owner filter inline in each resolver instead of pulling it into a helper?** A `withOwner(filters, ctx)` helper looks like the obvious refactor, but each resolver here is short, and the helper would barely save any lines. Writing the `owner` filter inline also keeps the rule visible at every read site, so a future reader of the resolver does not have to know that a helper exists. With twenty custom queries the helper would pay off; with three, inline is plainer.

## 6.4 Should we keep the GraphQL policy?

After Step 6, the `is-note-owner` policy from Step 4 is technically redundant. The Document Service middleware adds an `owner` filter to every `findOne`, so when `Mutation.updateNote` (or `togglePin`, `archiveNote`, `duplicateNote`) calls `strapi.documents("api::note.note").findOne(...)` to load the target note, a non-owner gets back nothing. The mutation fails before the policy would have had a chance to reject it.

The policy is still useful as a second check and as a worked example of how policies are written. Two reasonable choices:

- **Keep it.** A second check, run before the resolver. If someone disables or breaks the Document Service middleware by accident, GraphQL writes are still gated. The cost is that the same rule lives in two places.
- **Delete it.** One source of truth. Trust the Document Service middleware to do the work.

We keep the policy in this tutorial. The file is small, the overlap is on purpose, and the codebase ends up with two patterns side by side: a Document Service middleware doing the main work, plus a `policyContext`-based policy as a worked example.

## Step 7: Test the backend contract

The starter's test script already covers Part 2. Add a section at the bottom that covers ownership. Open `starter-template/scripts/test-graphql.mjs` (and the canonical copy in `server/scripts/test-graphql.mjs`) and append:

```

1 // 9. Ownership: two-user isolation
2 section("Ownership: two-user isolation");
3
4 const testuserJwt = (
5   await gql(
6     `mutation { register(input: { username: "testuser-${Date.now()}" }, email: "a-${Date.now()}" )
7   )
8   ).data?.register?.jwt;
9
10 const testuser2Jwt = (
11   await gql(
12     `mutation { register(input: { username: "testuser2-${Date.now()}" }, email: "b-${Date.now()}" )
13   )
14   ).data?.register?.jwt;
15
16 const testuserNote = (
17   await gql(
18     `mutation { createNote(data: { title: "testuser-only", content: "secret" }) { documentId`
19   )
20   ).data?.createNote?.documentId;

```



```

19     undefined,
20     { Authorization: `Bearer ${testuserJwt}` },
21   )
22  )?.data?.createNote;
23
24   const testuser2View = await gql(`{ notes { documentId } }`, undefined, {
25     Authorization: `Bearer ${testuser2Jwt}`,
26   });
27   check(
28     "testuser2 does not see testuser's notes in the list",
29     !testuser2View?.data?.notes?.some(
30       (n) => n.documentId === testuserNote.documentId,
31     ),
32   );
33
34   const testuser2Attack = await gql(
35     `mutation A($id: ID!) { togglePin(documentId: $id) { documentId } }`,
36     { id: testuserNote.documentId },
37     { Authorization: `Bearer ${testuser2Jwt}` },
38   );
39   check(
40     "testuser2 cannot toggle pin on testuser's note (Policy Failed)",
41     testuser2Attack?.errors?.some((e) =>
42       /Policy Failed|Forbidden/i.test(e.message),
43     ),
44   );

```

Run it:

```
npm run test:backend
```



**Heads-up: the existing checks now need authentication.** Sections 1 through 8 of `test-graphql.mjs` make anonymous GraphQL calls (no `Authorization` header). After Step 1.2 disabled anonymous reads on `Note`, the script bails at Section 1 with `No active notes in the database` and never reaches the new ownership block. Two ways to fix the script:

1. **Register a user at the top and authenticate every request** by passing `{ Authorization: \ Bearer ${jwt}` }` as the third argument to every `gql(...)`` call.
2. **Skip Sections 1 through 8** if you only want to run the ownership block; comment out everything before

```
// 5: Ownership, two user isolation.
```

The two new ownership checks themselves work correctly — they register fresh users and pass their own JWTs — so once you have `npm run test:backend` running again, those two lines will be green.

## Step 8: Wire up the frontend

Backend done. The frontend needs three things: pages to register and log in, a way to attach the JWT to every GraphQL call, and a route gate that sends signed-out users to `/login`.

### 8.1 Cookie helpers

Create `lib/auth.ts`:

```

1 // starter-template/lib/auth.ts
2 import { cookies } from "next/headers";

```



```

3
4 export const JWT_COOKIE = "strapi_jwt";
5
6 export async function getJwt(): Promise<string | undefined> {
7   const store = await cookies();
8   return store.get(JWT_COOKIE)?.value;
9 }
10
11 export async function setJwt(jwt: string): Promise<void> {
12   const store = await cookies();
13   store.set(JWT_COOKIE, jwt, {
14     httpOnly: true,
15     sameSite: "lax",
16     secure: process.env.NODE_ENV === "production",
17     path: "/",
18     maxAge: 60 * 60 * 24 * 30, // 30 days
19   });
20 }
21
22 export async function clearJwt(): Promise<void> {
23   const store = await cookies();
24   store.delete(JWT_COOKIE);
25 }

```

`httpOnly` matters. The browser keeps the cookie but JavaScript cannot read it, so an XSS payload cannot steal the token. The cookie is only read on the server, by Server Components and Server Actions through `cookies()`.

## 8.2 Attach the JWT in Apollo Client

Open `lib/apollo-client.ts` and add a fetch wrapper that adds the token when one is present:

```

1 // starter-template/lib/apollo-client.ts
2 import { HttpLink } from "@apollo/client";
3 import {
4   registerApolloClient,
5   ApolloClient,
6   InMemoryCache,
7 } from "@apollo/client-integration-nextjs";
8 import { getJwt } from "../auth";
9
10 const STRAPI_GRAPHQL_URL =
11   process.env.STRAPI_GRAPHQL_URL ?? "http://localhost:1337/graphql";
12
13 export const { getClient, query, PreloadQuery } = registerApolloClient(() => {
14   return new ApolloClient({
15     cache: new InMemoryCache({
16       typePolicies: {
17         Note: { keyFields: ["documentId"] },
18         Tag: { keyFields: ["documentId"] },
19         UsersPermissionsUser: { keyFields: ["id"] },
20       },
21     }),
22     link: new HttpLink({
23       uri: STRAPI_GRAPHQL_URL,
24       fetchOptions: { cache: "no-store" },
25       fetch: async (url, init = {}) => {
26         const jwt = await getJwt();
27         const headers = new Headers(init.headers);
28         if (jwt) headers.set("Authorization", `Bearer ${jwt}`);
29         return fetch(url, { ...init, headers });
30       },
31     }),

```

```
32   });  
33   });
```

`registerApolloClient` runs the factory once per request, so `getJwt()` runs fresh for every render and `Server Action`. The token never reaches the browser bundle.

### 8.3 Login, register, me operations

Append to `lib/graphql.ts`:

```
1  export const LOGIN = gql`  
2    mutation Login($input: UsersPermissionsLoginInput!) {  
3      login(input: $input) {  
4        jwt  
5        user {  
6          id  
7          username  
8          email  
9        }  
10     }  
11   }  
12 `;  
13  
14 export const REGISTER = gql`  
15   mutation Register($input: UsersPermissionsRegisterInput!) {  
16     register(input: $input) {  
17       jwt  
18       user {  
19         id  
20         username  
21         email  
22       }  
23     }  
24   }  
25 `;  
26  
27 export const ME = gql`  
28   query Me {  
29     me {  
30       id  
31       username  
32       email  
33     }  
34   }  
35 `;
```

### 8.4 The login page

The starter already ships `app/login/page.tsx` with the form, the error banner, and a stub `loginAction` that just `console.log`s. Open the file if you want to see the JSX. To wire it up, replace the contents of `app/login/actions.ts`:

```
1  // starter-template/app/login/actions.ts  
2  "use server";  
3  
4  import { redirect } from "next/navigation";  
5  import { getClient } from "@lib/apollo-client";  
6  import { LOGIN } from "@lib/graphql";  
7  import { setJwt } from "@lib/auth";
```

```

8
9 const asString = (v: FormDataEntryValue | null) =>
10   typeof v === "string" ? v : "";
11
12 export async function loginAction(formData: FormData) {
13   const identifier = asString(formData.get("identifier")).trim();
14   const password = asString(formData.get("password"));
15
16   if (!identifier || !password) return;
17
18   let jwt: string | undefined;
19   try {
20     const { data } = await getClient().mutate<{
21       login: { jwt: string };
22     }>({
23       mutation: LOGIN,
24       variables: { input: { identifier, password } },
25     });
26     jwt = data?.login?.jwt;
27   } catch {
28     // Apollo throws CombinedGraphQLErrors when Strapi returns an error
29     // (e.g. "Invalid identifier or password"). Catch it and fall through
30     // to the !jwt redirect below; otherwise the throw bubbles to Next's
31     // runtime overlay instead of giving the user an error message.
32   }
33
34   if (!jwt) redirect("/login?error=invalid");
35
36   await setJwt(jwt);
37   redirect("/");
38 }

```

Two things to notice:

- **The try/catch matters.** Apollo's `mutate()` throws on Strapi errors, it does not return `{ data: null }`. Without the catch, a wrong password produces a runtime error overlay in dev (and a 500 in production) instead of redirecting back to `/login?error=invalid`.
- **The page already handles `searchParams.error`.** The starter's `app/login/page.tsx` reads error from `searchParams` and renders a red banner when it equals `"invalid"` — that pairs with the redirect above.

The register page has the same shape with three inputs and the `REGISTER` mutation. The starter ships `app/register/page.tsx` and a stub `app/register/actions.ts` already; replace the action contents:

```

1 // starter-template/app/register/actions.ts
2 "use server";
3
4 import { redirect } from "next/navigation";
5 import { getClient } from "@lib/apollo-client";
6 import { REGISTER } from "@lib/graphql";
7 import { setJwt } from "@lib/auth";
8
9 const asString = (v: FormDataEntryValue | null) =>
10   typeof v === "string" ? v : "";
11
12 export async function registerAction(formData: FormData) {
13   const username = asString(formData.get("username")).trim();
14   const email = asString(formData.get("email")).trim();
15   const password = asString(formData.get("password"));
16
17   if (!username || !email || !password) return;
18

```

```

19   let jwt: string | undefined;
20   try {
21     const { data } = await getClient().mutate<{
22       register: { jwt: string };
23     }>({
24       mutation: REGISTER,
25       variables: { input: { username, email, password } },
26     });
27     jwt = data?.register?.jwt;
28   } catch {
29     // Same try/catch shape as the login action: Apollo throws on Strapi
30     // validation errors (duplicate username/email, password too short),
31     // so we catch and fall through to the redirect below.
32   }
33
34   if (!jwt) redirect("/register?error=invalid");
35
36   await setJwt(jwt);
37   redirect("/");
38 }

```

## 8.5 Logout

The starter has `app/logout/actions.ts` as a stub. Replace its contents:

```

1 // starter-template/app/logout/actions.ts
2 "use server";
3
4 import { redirect } from "next/navigation";
5 import { clearJwt } from "@/lib/auth";
6
7 export async function logoutAction() {
8   await clearJwt();
9   redirect("/login");
10 }

```

Replace `components/nav.tsx` with a version that reads the signed-in user via the `me` query and shows their username plus a sign-out button when authenticated. When anonymous (e.g. on `/login` or `/register`), it falls back to a "Sign in" link.

```

1 // components/nav.tsx
2 import Link from "next/link";
3 import { query } from "@/lib/apollo-client";
4 import { ME } from "@/lib/graphql";
5 import { logoutAction } from "@/app/logout/actions";
6
7 const LINKS = [
8   { href: "/", label: "Notes" },
9   { href: "/search", label: "Search" },
10  { href: "/stats", label: "Stats" },
11 ];
12
13 type Me = { id: string; username: string; email: string } | null;
14
15 export async function Nav() {
16   // The Apollo client (Step 8.2) injects the JWT cookie automatically.
17   // On /login and /register there is no JWT, so the query throws and we
18   // fall back to anonymous.
19   let me: Me = null;

```

```

20   try {
21     const { data } = await query<{ me: Me }>({ query: ME });
22     me = data?.me ?? null;
23   } catch {
24     me = null;
25   }
26
27   return (
28     <header className="border-b">
29       <div className="mx-auto flex max-w-3xl items-center justify-between px-6 py-4">
30         <Link href="/" className="text-lg font-semibold">
31           Notes
32         </Link>
33         <nav className="flex items-center gap-5 text-sm text-neutral-600">
34           {me ? (
35             <>
36               {LINKS.map((l) => (
37                 <Link key={l.href} href={l.href} className="hover:text-black">
38                   {l.label}
39                 </Link>
40               ))}
41               <Link
42                 href="/notes/new"
43                 className="rounded bg-black px-3 py-1.5 text-sm font-medium text-white f
44             >
45               New
46             </Link>
47             <span className="text-neutral-500">@{me.username}</span>
48             <form action={logoutAction}>
49               <button
50                 type="submit"
51                 className="text-neutral-500 hover:text-black"
52             >
53               Sign out
54             </button>
55             </form>
56           </>
57           ) : (
58             <Link
59               href="/login"
60               className="rounded bg-black px-3 py-1.5 text-sm font-medium text-white hov
61             >
62               Sign in
63             </Link>
64           )}
65         </nav>
66       </div>
67     </header>
68   );
69 }

```

Four things to notice:

- **Nav is now async**. It runs as a Server Component, calls `me`, and renders the result. There is no client-side fetch; the user state is resolved on the server before the HTML ships.
- **The `try/catch` is required**. When the request has no JWT cookie (the `/login` and `/register` routes), the `me` query rejects with a `Forbidden` error from Apollo. Catching it and falling back to `me = null` keeps the public pages rendering normally.
- **The whole nav switches on auth**. When `me` is null (anonymous), only the brand link and the **Sign in** button render. Notes / Search / Stats / New are all behind sign-in, both because the routes are protected by



`middleware.ts` (Step 8.6) and because there is nothing meaningful to point at when the user has no notes yet.

- The sign-out button is a `<form action={logoutAction}>`. That is the supported shape for invoking a Server Action without args. The button submits, the action runs, the cookie is cleared, and the redirect to `/login` happens server-side.

## 8.6 Protect the routes with Next.js Proxy

**Next 16 rename.** What was called Middleware in earlier versions of Next.js is now called Proxy. The functionality is identical; only the file name ( `proxy.ts` instead of `middleware.ts` ) and the exported function name ( `proxy` instead of `middleware` ) changed. See `node_modules/next/dist/docs/01-app/01-getting-started/16-proxy.md` for the official note.

Create `proxy.ts` at the root of the starter (next to `package.json`, not inside `app/`):

```
1 // starter-template/proxy.ts
2 import { NextResponse, type NextRequest } from "next/server";
3 import { JWT_COOKIE } from "@/lib/auth";
4
5 const PUBLIC_ROUTES = ["/login", "/register"];
6
7 export function proxy(req: NextRequest) {
8   const { pathname } = req.nextUrl;
9
10  if (PUBLIC_ROUTES.some((p) => pathname.startsWith(p))) {
11    return NextResponse.next();
12  }
13
14  const hasJwt = req.cookies.has(JWT_COOKIE);
15  if (!hasJwt) {
16    const url = req.nextUrl.clone();
17    url.pathname = "/login";
18    url.searchParams.set("returnTo", pathname);
19    return NextResponse.redirect(url);
20  }
21
22  return NextResponse.next();
23 }
24
25 export const config = {
26   matcher: ["/((?!_next/|favicon|api/).*)"],
27 };

```

This uses Next.js's `matcher` syntax. The pattern excludes Next.js's internal `_next/` paths, the favicon, and any `/api/` routes you might add later. Everything else requires a JWT.

## 8.7 A clearer 404 for the detail route

The Document Service middleware from Step 6 makes other users' notes invisible: when `testuser2` fetches `testuser's` note by `documentId`, the lookup misses, the Server Component calls `notFound()`, and the user sees Next's default 404 – This page could not be found. page. That is the security-correct behavior: leaking "this note exists but isn't yours" would tell an attacker probing IDs which ones belong to other users versus which ones don't exist.

The starter ships a generic `app/notes/[documentId]/not-found.tsx` page that catches the `notFound()` call and renders a friendlier message than Next's default. Replace its contents with a tutorial-aware version that names the policy doing the work, so a reader trying the cross-user URL trick can see the system is working as designed:



```

 9 |                                     read scoping on findMany/findOne,
10 |                                     write gating on update/delete)
11 | └─ api/
12 |   └─ note/
13 |     └─ content-types/
14 |       └─ note/
15 |         └─ schema.json # updated: owner relation
16 | └─ policies/
17 |   └─ cap-page-size.ts
18 |   └─ is-note-owner.ts # new: GraphQL write policy
19 | └─ extensions/
20 |   └─ graphql/
21 |     └─ index.ts
22 |     └─ middlewares-and-policies.ts
23 |     └─ computed-fields.ts
24 |     └─ queries.ts # updated: ownership filter in
25 |                   searchNotes / noteStats / notesByTag
26 |     └─ mutations.ts # updated: auth.scope on the three
27 |                   custom mutations

```

The frontend:

```

 1 starter-template/
 2 └─ proxy.ts # new: route protection
 3 |           (Next.js 16 rename of middleware.ts)
 4 └─ app/
 5 |   └─ login/
 6 |     └─ page.tsx # new
 7 |     └─ actions.ts # new
 8 |   └─ register/ # new (same shape as login/)
 9 |   └─ logout/
10 |     └─ actions.ts # new
11 |   └─ notes/ # unchanged from Part 3
12 └─ lib/
13 |   └─ apollo-client.ts # updated: JWT fetch wrapper
14 |   └─ auth.ts # new: cookie helpers
15 |   └─ graphql.ts # updated: LOGIN, REGISTER, ME

```

## What you just built

- **Strapi authentication** through the `users-permissions` plugin: register, login, JWT issuance, and `ctx.state.user` populated on every authenticated request.
- **A Note → User ownership relation with automatic owner stamping** via a Document Service middleware in `src/index.ts` `register()`. Clients cannot claim someone else's identity, regardless of whether they hit REST, GraphQL, or any other code path that reaches the Document Service.
- **Read scoping for both REST and GraphQL** via a second clause in the same Document Service middleware. It injects `owner = ctx.state.user.id` into the filter on every `findMany` and `findOne` against `api::note.note`. One file, one rule, both API surfaces covered.
- **Write gating for both REST and GraphQL** via a third clause in the same middleware. On `update` and `delete` it does a pre-flight `findOne` (which inherits the read filter from the second clause) and throws `NotFoundError` when the row is not visible to the caller. REST `PUT /api/notes/:id` and GraphQL `Mutation.updateNote` are both covered without any per-resolver wiring.
- **An `is-note-owner` policy** attached to four GraphQL write mutations as a worked example of authorization-shaped logic in `policyContext` and as a second check at the resolver boundary.
- **Per-resolver scoping on the three custom queries** (`searchNotes`, `noteStats`, `notesByTag`), since `noteStats` calls `count()` and `findMany` on `api::tag.tag`, neither of which the Document Service

middleware matches; and because writing the `owner` filter inline keeps the rule visible at the call site for `searchNotes` and `notesByTag` even though it would otherwise merge in via the middleware.

- **A frontend authentication flow** with HTTP-only cookie-stored JWT, `/login` and `/register` Server Actions, an Apollo fetch wrapper that attaches the token, and a Next.js Proxy ( `proxy.ts` ) that redirects unauthenticated requests to `/login?returnTo=...`
- **A second pass of the test script** with a two-user isolation assertion that exercises the ownership model end to end.

## The ownership pattern, generalized

Stripped of the note-taking specifics, the pattern is:

1. **Add an owner relation to every owner-scoped content type.** `manyToOne User, named owner`.
2. **Stamp the owner automatically with a Document Service middleware** in `src/index.ts` `register()`.  
Read `strapi.requestContext.get()?.state?.user`, assign to `params.data.owner` on `create`. This closes the "client claims any owner" hole below the API surface, so it covers REST, GraphQL, and any other code path that calls into the Document Service.
3. **Scope reads with a second Document Service middleware** in the same file. On `findMany` and `findOne` for that content type, add `owner = ctx.state.user.id` to `params.filters`. Both REST controllers and GraphQL Shadow CRUD resolvers go through the Document Service, so one rule covers both APIs.
4. **For each custom resolver that does not flow through Shadow CRUD, add the owner filter inline.** Custom queries that build their own filter object or query a different content type bypass the Document Service middleware's rule and have to scope themselves.
5. **Optionally add a per-resolver policy on GraphQL writes** as a second check at the resolver boundary, and as a place to express GraphQL-shaped business rules ("you can only archive a note that has at least one tag" is awkward to put in a Document Service middleware).

Step 1 is the data model. Step 2 is the create-time invariant. Step 3 is read scoping. Step 4 is per-resolver coverage for anything Step 3 misses. Step 5 is taste.

Every Strapi project that needs per-user data ends up here. The `users-permissions` plugin handles authentication; you write authorization as a small set of Document Service middlewares plus, optionally, a policy or two for GraphQL writes.

## What's next

This is the end of the four-part series. The repo at this point is a complete Strapi + Next.js application with auth, ownership, soft-delete, GraphQL customization, and a frontend that exercises everything.

Areas worth exploring on your own from here:

- **Sharing notes between users.** Add a many-to-many `sharedWith` relation on `Note`. Update the read-scoping Document Service middleware to allow reads when `owner.id === user.id || sharedWith` includes the user. Writes still gate to owner only.
- **Roles beyond owner / not-owner.** Add an `Admin` role in `users-permissions`. The read-scoping middleware can short-circuit with `return next()` for admin users; everyone else goes through the owner filter.
- **Refresh tokens.** Issue a short-lived access token alongside the JWT, store the refresh token in a separate HTTP-only cookie, rotate on every authenticated request.
- **Real-time updates.** Add a Document Service middleware on `update` / `create` / `delete` that publishes to a pub/sub layer (Postgres LISTEN/NOTIFY, Redis, or a managed broker), and surface the events as GraphQL subscriptions on the Apollo side. The ownership filter from Step 6 is already authoritative, so subscriptions inherit it.

Each of those builds on the same shape: a content-type relation plus one or more Document Service middlewares. The mechanics from this series are the load-bearing parts.

If you found this series useful, the [Strapi blog](#) has more deep dives in this style, and the [Strapi v5 docs](#) are the canonical reference for everything we used here. Thanks for reading.

Paul Bratslavsky

Developer Advocate

[Start Discussion](#)

0 replies

**Build modern websites without sacrificing customization in minutes instead of days**

```
npx create-strapi-app@latest
```

[Copy](#)

**Open source (MIT)**

**SOC 2 certified**

**GDPR Compliant**

### Ready to deploy?

Ready to deploy? Start building with a free plan and scale as needed, fast and simple.

[Start Deploying →](#)

### Explore Strapi Enterprise

Explore Strapi Enterprise with an interactive product tour, trial, or a personalized demo.

[Explore Enterprise →](#)



Strapi is the leading open-source Headless CMS. Strapi gives developers the freedom to use their favorite tools and frameworks while allowing editors to easily manage their content and distribute it anywhere.



| PRODUCT            | SOLUTIONS                   | RESOURCES            | INTEGRATIONS     | COMPANY      |
|--------------------|-----------------------------|----------------------|------------------|--------------|
| Strapi 5           | Modern Websites             | Strapi CMS docs      | All integrations | About us     |
| Strapi Cloud       | Mobile applications         | Strapi Cloud Docs    | React CMS        | Blog         |
| Enterprise Edition | Ecommerce                   | Partner Directory    | Next.js CMS      | Handbook     |
| Features           | Backend Framework           | Strapi Events        | Tanstack CMS     | Careers      |
| Strapi AI          | Learning Management System  | Tutorials            | Vue.js CMS       | Contact      |
| Roadmap            | Intranet CMS                | GitHub repository    | Nuxt.js CMS      | FAQ          |
| Changelog          | Product Information Systems | Case Studies         | Astro CMS        | Newsroom     |
| Plugin Marketplace | For developers              | Strapi vs Wordpress  | Flutter CMS      | Goodies shop |
| Try live demo      | For content teams           | Strapi vs Contentful | Svelte CMS       |              |
|                    | For business teams          | Video Tutorials      | React Native CMS |              |
|                    | For Digital Agencies        |                      |                  |              |

---

Join us on

© 2026, Strapi [License](#) [Terms](#) [Privacy](#).